ESTRATEGIA DE ESCALABILIDADE

1. INTERPRETAÇÃO DO CENÁRIO

Ponto	Decisão adotada
Período	1 milhão de escritas e 2 milhões de leituras por dia, num dia de movimento alto (sexta ou sábado)
Proporção	1 escrita para cada 2 leituras (33% do tráfego é escrita)
Concentração no pico	30% do dia entre 17h e 20h, mesmo critério do mapeamento
Distribuição entre os fluxos	Mesma participação de cada fluxo encontrada no mapeamento
Requisição de escrita	Qualquer operação que grava: ação do usuário, webhook do gateway e tarefa do worker (cada aviso gravado conta como uma escrita)

Por que a proporção 1:2 importa: na maioria dos aplicativos as leituras são 10 ou mais vezes maiores que as escritas. Com 1:2, um terço do tráfego é escrita, e toda escrita vai para um único lugar, o banco principal. Réplicas e cache ajudam só as leituras. Por isso as decisões deste documento se concentram em deixar cada escrita mais barata.

2. VOLUME POR FLUXO

2.1 Escritas (1.000.000 por dia)

Fluxo	Participação	Escritas/dia	Pico (escritas/s)
Notificações	88,5%	884.600	24,6
Candidatura/contratação	5,2%	51.600	1,4
Chat	5,2%	51.600	1,4
Pagamento	1,0%	10.000	0,3
Busca de vagas (favoritos)	0,2%	2.200	0,1
Total	100%	1.000.000	27,8

2.2 Leituras (2.000.000 por dia)

Fluxo	Participação	Leituras/dia	Pico (leituras/s)
Busca de vagas	54,3%	1.086.800	30,2
Notificações	15,1%	301.900	8,4
Chat	15,1%	301.900	8,4
Candidatura/contratação	13,6%	271.700	7,5
Pagamento	1,9%	37.700	1,0
Total	100%	2.000.000	55,6

Pico = volume do dia x 30% / 10.800 s (3 horas). No pico, o sistema recebe cerca de 83 requisições por segundo, sendo 28 de escrita e 56 de leitura.

2.3 Rajadas

A média esconde o pior momento. A publicação de uma vaga grava de uma vez os avisos de todos os freelancers elegíveis (1.200 no cenário inicial), e o RNF15 dá 5 s para isso. Se 10 vagas forem publicadas no mesmo minuto às 17h, são 12 mil avisos para gravar em poucos segundos. A arquitetura precisa aguentar essa rajada, não só a média de 28 escritas por segundo.

3. FLUXOS DE MAIOR CARGA

Posição	Fluxo	Por que pesa neste cenário
1	Notificações	88,5% das escritas, chegando em rajadas, e 15% das leituras (contador de não lidas)
2	Busca de vagas	54% das leituras, todas no horário de pico, com prazo de 3 s (RNF08)
3	Chat	5% das escritas e 15% das leituras, com tempo real esperado pelo usuário
4	Candidatura/contratação	Volume menor, mas cada escrita mexe em várias tabelas e há disputa pela mesma vaga
5	Pagamento	Menor volume, mas não pode ter erro (consistência acima de desempenho)

Notificações e busca juntas somam cerca de 90% das escritas e 70% das leituras. As estratégias abaixo começam por elas.

4. ESTRATÉGIAS DEFINIDAS

4.1 Filas

Decisão	Detalhe
Fila de tarefas em tabela do PostgreSQL	Tabela fila_tarefa, lida pelo worker com FOR UPDATE SKIP LOCKED e acordada por LISTEN/NOTIFY
O que passa pela fila	Avisos de vaga nova e substituição, push e e-mail, webhook do pagamento, liberação por prazo, auditoria
Workers	2 no cenário (um assume se o outro cair); mais cópias se a fila atrasar
Gravação dos avisos em lote	Um INSERT ... SELECT grava todos os avisos de uma vaga num só comando
Push em lote	Até 500 aparelhos por chamada ao Serviço de Notificações

Justificativa para o cenário:

- 88,5% das escritas são avisos. Se fossem gravados na requisição da empresa, cada publicação de vaga esperaria milhares de gravações. Pela fila, a empresa recebe a resposta na hora e o worker absorve a rajada.
- Gravar em lote transforma 1.200 comandos em 1. No pico de 10 vagas por minuto, são 10 comandos grandes em vez de 12 mil pequenos, o que cabe nos 5 s do RNF15.
- A fila fica no próprio PostgreSQL porque a tarefa é gravada na mesma transação da vaga (padrão outbox): não existe vaga publicada sem aviso. O volume (menos de 30 escritas por segundo em média no pico) não justifica instalar RabbitMQ ou Kafka.

4.2 Índices

Índice	Fluxo
vaga (funcao_id, inicio) WHERE status = 'PUBLICADA'	Busca de vagas
notificacao (usuario_id, criada_em DESC) WHERE lida = FALSE	Notificações
mensagem (conversa_id, enviada_em)	Chat
candidatura (vaga_id, status)	Candidatura
contratacao (status, confirmada_em)	Contratação e pagamento
freelancer_funcao (funcao_id, freelancer_id)	Avisos e substituição

Justificativa para o cenário:

- Com 1:2, cada índice tem um custo alto: toda escrita precisa atualizar todos os índices da tabela. Por isso a regra é poucos índices nas tabelas mais escritas. A notificacao, que recebe 88,5% das escritas, tem só a chave e um índice parcial.
- Os índices parciais guardam só o que a consulta procura (vagas publicadas, avisos não lidos). Ficam pequenos, cabem na memória do servidor e custam menos a cada escrita.
- Marcar como lida é feito em lote: ao abrir a lista, um único UPDATE marca todos os avisos do usuário, em vez de um por aviso.

4.3 Réplicas de leitura

Decisão	Detalhe
Quantidade	1 réplica, replicação em streaming assíncrona do PostgreSQL 16
Vai para a réplica	Busca de vagas, detalhe de vaga, perfis, lista de candidatos, avaliações, histórico antigo do chat
Fica no banco principal	Pagamento, carteira, regras que conferem antes de gravar, tela logo após o próprio usuário gravar, substituição

Decisão	Detalhe
Quando acrescentar a 2ª	Se a réplica passar de 70% de CPU no pico ou o atraso dela passar de 1 s

Justificativa para o cenário:

- Com 1:2, o banco principal já recebe um terço do tráfego só de escritas. Tirar dele a busca, que é mais da metade das leituras, deixa folga para as rajadas de avisos.
- O atraso normal da réplica é menor que 1 s, dentro dos 5 s que o RNF15 dá para a vaga nova aparecer.
- A réplica também é o plano de recuperação: se o principal cair, ela é promovida em minutos. Só o backup diário perderia até um dia de dados.
- Leituras de dinheiro e de regra ficam no principal porque um dado atrasado ali causa erro real, como saldo errado ou duas contratações para a mesma vaga.

4.4 Cache

O que fica no cache (Redis)	Validade	Efeito no cenário
Primeira página da busca por função e cidade	60 s, apagada quando vaga da função é publicada ou preenchida	Corta cerca de metade das leituras da busca (estimativa)
Resposta do Serviço de Mapas por CEP	30 dias	Evita chamada externa de até 2 s (RNF26) em cada busca
Contador de não lidas por usuário	Atualizado pelo worker ao gravar avisos; apagado ao marcar como lidas	Tira do banco boa parte das 302 mil leituras de notificação
Contadores do limite de requisições (RNF30)	1 min	Limite único para todos os servidores de aplicação

FREELAS

Não vão para o cache: carteira, pagamento, status de candidatura e contratação, mensagens do chat.

Justificativa para o cenário:

- A busca é a maior leitura (1,09 milhão por dia) e concentra-se no mesmo horário, com muitos freelancers da mesma cidade abrindo a mesma página. É o caso em que o cache mais ajuda.
- O contador de não lidas é lido toda vez que o app abre, mas só muda quando chega aviso ou o usuário lê. Guardar o número pronto evita uma consulta ao banco a cada abertura.
- O app continua funcionando se o Redis cair: o servidor vai direto ao banco (cache-aside). Nenhum dado oficial existe só no cache.

4.5 Particionamento e retenção

Tabela	Partição	Retenção
notificacao	Por mês	90 dias, depois a partição é apagada
log_auditoria	Por mês	Prazo definido pelo grupo para o RNF04, depois vai para o armazenamento de arquivos

Justificativa para o cenário: com cerca de 885 mil avisos por dia, a notificacao cresceria em torno de 26 milhões de linhas por mês. Estimando 300 bytes por linha com índices, são cerca de 8 GB por mês. Com retenção de 90 dias e partições, a tabela fica estável perto de 24 GB, e apagar um mês inteiro é instantâneo. Sem isso, a tabela cresce sem parar e os DELETE de limpeza competem com as escritas do pico.

4.6 Concorrência e pool de conexões

Situação	Técnica
Várias contratações para a mesma vaga	SELECT ... FOR UPDATE na vaga
Vários aceites na mesma substituição	UNIQUE em substituicao.nova_contratacao_id (já existe)
Crédito e saque na carteira	SELECT ... FOR UPDATE na carteira, com movimentacao_carteira na mesma transação
Webhook repetido	UNIQUE em chave_idempotencia (já existe)
Muitas conexões no pico	PgBouncer em modo transação entre aplicação/worker e PostgreSQL

Justificativa: as transações são curtas, então o bloqueio dura milissegundos e não forma fila mesmo no pico. O PgBouncer evita que servidores de aplicação, workers e réplica abram centenas de conexões, que no PostgreSQL custam um processo cada.

FREELAS

5. INFRAESTRUTURA PARA O CENÁRIO

Peça	Quantidade e tamanho (estimativa)	Papel no cenário
Balanceador de carga	1 (serviço gerenciado)	Distribui as 83 requisições/s do pico
Servidores de aplicação	3 cópias stateless, 2 vCPU e 4 GB cada, com escala automática	Uma cópia pode cair ou ser atualizada sem afetar o pico
Worker	2 cópias	Fila de avisos, push, webhook e prazos
PostgreSQL 16 principal	1 servidor, 4 a 8 vCPU, 32 GB de memória, disco SSD	Todas as escritas e leituras críticas
PostgreSQL 16 réplica	1 servidor do mesmo tamanho	Busca, perfis e substituto em caso de falha
PgBouncer	1 por servidor de banco	Pool de conexões
Redis	1 instância gerenciada, 1 a 2 GB	Cache e limite de requisições
Backup	Diário, mais arquivamento contínuo do log do banco (WAL)	Recuperar até o último minuto, não só até a última cópia diária

Carga estimada no pico depois das estratégias:

Destino	Escritas/s	Leituras/s	Observação
Redis	-	~21	Metade da busca e a maior parte dos contadores de não lidas
Réplica	-	~19	Restante da busca, perfis, lista de candidatos
PostgreSQL principal	~28 (avisos em lote)	~16	Chat, candidatura, pagamento, regras

Esses números ficam muito abaixo da capacidade de um PostgreSQL 16 nesse tamanho de servidor, que é da ordem de milhares de transações simples por segundo. A folga existe de propósito: ela absorve as rajadas de avisos e o crescimento. O número exato deve ser confirmado com teste de carga (por exemplo, pgbench e uma ferramenta de carga HTTP) antes de ir para produção.

6. DECISÕES JUSTIFICADAS PARA O VOLUME

Decisão	Por que neste cenário
Um único banco principal, sem dividir os dados em vários bancos (sharding)	28 escritas/s no pico, ou cerca de 280/s se o volume fosse por hora, cabem num só servidor. Dividir o banco quebraria as transações que juntam vaga, candidatura e contratação, e daria muito trabalho sem necessidade.
Crescer primeiro aumentando o servidor	Aumentar CPU e memória do principal não muda nada no código. É o caminho mais simples enquanto houver tamanho maior disponível.
Se um dia o principal não der conta, separar só as notificações	Elas são 88,5% das escritas e não participam de transações com as outras tabelas. Mover notificacao para um banco próprio alivia o principal sem mexer no resto do modelo.
Fila no PostgreSQL, sem serviço de fila separado	O outbox garante que vaga e aviso são gravados juntos, e o volume não exige outro produto.
Réplica assíncrona, não síncrona	A síncrona faria toda escrita esperar a réplica confirmar, o que encarece justamente a operação que mais pesa no cenário 1:2.
Cache só onde o dado tolera alguns segundos de atraso	Busca e contadores aceitam atraso curto. Dinheiro e status não aceitam.
Poucos índices nas tabelas mais escritas	Cada índice a mais é uma gravação a mais em cada uma das 885 mil escritas diárias de aviso.

Sinais para acionar o próximo passo de escala:

Sinal	Ação
Tarefa mais antiga da fila com mais de 5 s	Subir mais cópias do worker (RNF15 em risco)
CPU do principal acima de 70% no pico	Aumentar o servidor; depois, separar notificacao
Atraso da réplica acima de 1 s ou CPU acima de 70%	Acrescentar uma 2ª réplica
Busca acima de 3 s (RNF08)	Revisar o índice da busca e a validade do cache